

Temario

- Procesos y threads. Comunicación entre procesos.
- Funciones POSIX para acceso al sistemas de archivos (abstracción open, read, write, close).
- Drivers de dispositivos de E/S.

Software y Hardware

El parcial se realiza sobre las computadoras de laboratorio. Utilizar la versión de Linux de los laboratorios, y la versión de Xinu que está en la web de la materia. Sólo se puede utilizar la web de la materia, PEDCO, editor pluma y las herramientas de desarrollo. NO se puede utilizar servicios de google, drive, chat, etc.

1. Desarrollar un programa en **LINUX** para ordenar un arreglo con dos threads.

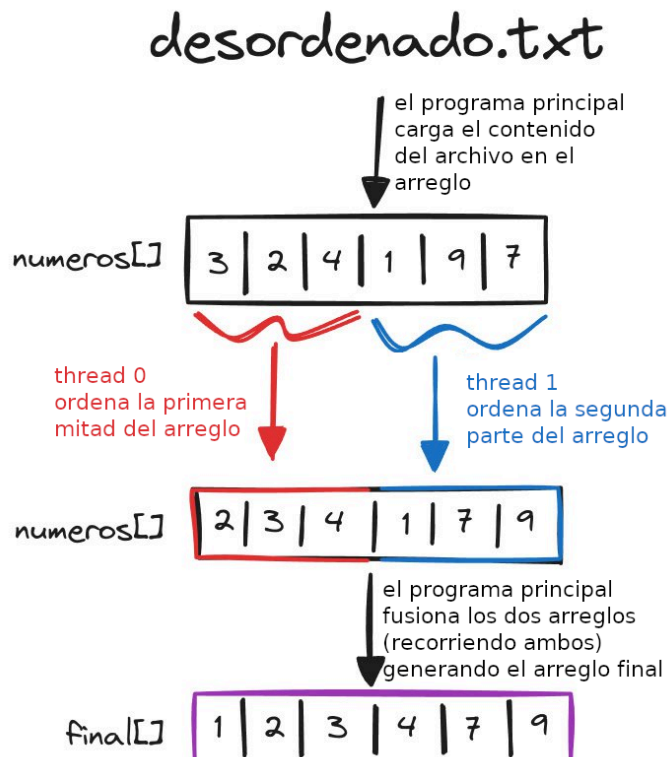
Existe una lista de 1000 números enteros en el archivo desordenado.txt. (NOTA: el archivo no es de texto, es de datos binarios).

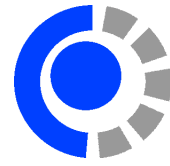
Desarrollar un programa que cargue los números del archivo en un único arreglo de enteros.

Luego, la mitad del arreglo debe ser ordenado de menor a mayor por un thread, y la segunda mitad del arreglo debe ser ordenado de menor a mayor por un segundo thread.

Luego de que los dos threads finalizaron, el proceso principal hará una fusión de las dos mitades en un segundo arreglo final del mismo tamaño. En este arreglo final la lista quedará ordenada luego de fusionada las dos mitades.

Al finalizar, este proceso muestra cada número del arreglo final ordenado por pantalla (cada número en una fila diferente).





Para leer un entero desde el archivo se puede usar `sizeof` de la variable entera. Por ejemplo, este código leerá el primer entero desde el archivo:

```
int fd, n;
....
// una vez abierto, leemos un entero desde el archivo
read(fd, &n, sizeof(n));
printf("nro: %i \n", n);    // y lo muestra
```

2. Desarrollar tres programas en **XINU** (tres procesos) que verifican si un número es primo y/o es un año bisiesto.

El proceso principal, debe esperar una entrada del usuario, y luego, debe utilizar algún mecanismo de comunicación apropiado para enviar la entrada del usuario a los procesos restantes.

El proceso 2 verifica si el número es primo y reporta el resultado al proceso principal.

El proceso 3 verifica si el número es un año bisiesto y reporta el resultado al programa principal.

Ambos procesos deben reportar el resultado usando el mismo mecanismo de comunicación que usó el programa principal.

El programa principal reporta si el número ingresado por el usuario es primo y/o año bisiesto.

Para leer una entrada del usuario en la CONSOLA y convertirlo en un número entero (las entradas en la consola son "cadenas de texto"), aquí hay un código de ayuda:

```
#include <xinu.h>

...etc...
..... en el programa donde se deba leer la entrada del usuario ....

int i;
char buf[10];

printf("INGRESE UN NRO: ");

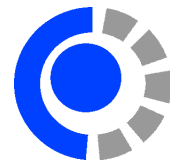
read(CONSOLE, buf, 10);
i = atoi(buf);

printf ("el numero ingresado entero es %d \n", i);

/* atoi() : ascii to int es una función de C.
 * XINU la tiene en lib/. Esta funcion sirve
 * para convertir una cadena ASCII que tiene
 * un numero con simbolos asciis a un entero (int)
 */
```



Sistemas Operativos I 2024
Segundo parcial



3. Responder:

- a. Defienda o critique la siguiente frase (justificar su postura): Una rutina de atención de interrupciones puede bloquear, por ejemplo, al tener que accionar algún hardware y esperando a que responda.
- b. ¿Considera que el programa `convertir_a_gris.c` original visto en la práctica, podría ser portado sin mucho esfuerzo a MAC OS? ¿O a FreeBSD?. Si/no, porqué. Justifique su respuesta.
- c. Defienda o critique la frase. Justificar: En XINU los procesos no pueden utilizar el mecanismo de comunicación inter procesos “memoria compartida”, porque el sistema operativo no provee tal funcionalidad.
- d. Defienda o critique la siguiente declaración: En un driver de dispositivo de E/S utilizar un buffer (como el implementado en la práctica) no tiene mucho sentido en realidad, ya que un proceso puede inmediatamente consumir una entrada (o generar una salida) cuando el driver obtiene un dato un de entrada via una interrupción (o emitió un dato de salida).
- e. Con lo aprendido durante el cursado, ¿considera que los desarrolladores de Android (*) habrán tenido en cuenta POSIX?. ¿Por qué lo harían?. Justifique su respuesta.
(*) (Google, quien posiblemente su principal misión es generar ingresos, y podrían o no querer ser compatibles con otros sistemas).